

React moderno na prática

Módulo: 09

Aula: 10 - Testes com Jest e React Testing Library + prática no projeto Instrutor: Lucca Dumas

CONTEÚDO

1) Abertura da aula

Na aula anterior, consolidamos o fluxo **tela -> serviço -> tabela** e a **navegação** no **frontend**. Nesta aula, adicionamos uma camada de **confiança no código**: testes automatizados no **mesmo repositório** que vocês já usam, com as mesmas ferramentas do time.

Objetivo central:

- entender o **papel** do teste unitário no front;
- localizar no projeto **Jest** e **React Testing Library**;
- ler e, em seguida, **escrever** um teste alinhado ao padrão existente;
- voltar à **prática em tela** para reforçar o conteúdo do curso.

2) O que veremos hoje?

1. Onde entram testes no fluxo de entrega (visão de produto)
2. Jest e React Testing Library no **frontend** (scripts, pastas)
3. Estrutura de um teste: arranjo, ação, asserção
4. Exemplo de referência no repositório (**Button**)
5. Prática guiada + continuidade da aplicação no projeto

3) Testes no front: o que muda

No backend, o teste unitário costuma focar em **funções puras** e regras. No React, o “unitário” muitas vezes significa **testar um componente** como unidade: o que aparece na tela e o que acontece com clique ou digitação.

Ferramentas usadas no **frontend** (ver `package.json`):

- **Jest** — executa os testes, `describe / it`, `matchers (expect)`
- **React Testing Library** — `render`, `screen`, busca por texto/role (como o usuário vê)
- **@testing-library/jest-dom** — `matchers` como `toBeInTheDocument()`
- **Ambiente** — `jest-environment-jsdom` simula o DOM no Node

Comandos usuais:

```
yarn test
yarn test:watch
```

A configuração fica em `jest.config.js` (raiz do **frontend**).

4) Onde ficam os testes no projeto

Padrão adotado:

```
src/
  __tests__/
    components/
    screens/
    hooks/
    ...
```

O Jest encontra arquivos que seguem o padrão (por exemplo `**/__tests__/**/*.test.tsx`), conforme `testMatch` em `jest.config.js`.

5) Estrutura mental de um teste

1. **Arranjo** — montar o componente na tela de teste (`render`)
2. **Ação** — simular interação (`fireEvent` ou `userEvent` , conforme o padrão do arquivo)
3. **Asserção** — verificar resultado (`expect`)

Exemplo de leitura (vocalização do time: descrições em português):

```
import { render, screen, fireEvent } from '@testing-library/react';
import { describe, it, expect } from '@jest/globals';
import '@testing-library/jest-dom';
import Button from '@components/buttons/button';

describe('Button', () => {
  it('deve renderizar o botão com o texto informado', () => {
    render(<Button>Salvar</Button>);
    expect(screen.getByText('Salvar')).toBeInTheDocument();
  });

  it('deve chamar a função ao clicar', () => {
    const aoClicar = jest.fn();
    render(<Button onClick={aoClicar}>Enviar</Button>);
    fireEvent.click(screen.getByText('Enviar'));
    expect(aoClicar).toHaveBeenCalledTimes(1);
  });
});
```

Referência completa no repositório: `src/__tests__/components/buttons/button.test.tsx`.

O que cada função faz

- `describe('Button', ...)` agrupa testes do mesmo componente.
- `it('...', ...)` define um cenário de teste.
- `render(<Button />)` monta o componente em uma tela de teste.
- `screen.getByText('Salvar')` procura um elemento pelo texto visível.
- `fireEvent.click(...)` simula o clique do usuário.
- `jest.fn()` cria uma função falsa para validar chamadas.
- `expect(...)` inicia a verificação do resultado esperado.
- `toBeInTheDocument()` confirma que o elemento existe na tela.
- `toHaveBeenCalledTimes(1)` confirma quantas vezes a função foi chamada.

6) O que priorizar no primeiro contato

- testar **comportamento visível** (texto, botão desabilitado, classes esperadas quando fizer parte do contrato de UI)
- usar **papel** (`getByRole`) ou **texto** (`getByText`) quando fizer sentido
- evitar, no início, acoplar o teste a detalhes internos de implementação que mudam o tempo todo
- funções e dados remotos: costuma-se **mocar** o serviço (outro tópico; nesta aula o foco é o fluxo de teste no componente)

7) Conexão com a aula anterior (prática)

Retomar o fio condutor:

1. abrir `src/screens/Administracao/Usuarios/index.tsx`
2. percorrer **navegação** (`useNavigate` -> cadastro)
3. percorrer **request** (`usuariosEndpoints.pesquisarUsuarios`)
4. listar o que seria **candidato a teste** (presença do título, botão Adicionar, `data-testid` já usado na tela) — sem obrigar implementação de mock de API na primeira rodada

Prática recomendada:

- executar o teste do `Button` com `yarn test button.test`
- adicionar **um** `it` em arquivo de teste existente, ou escrever um teste enxuto para componente já usado na tela

8) Hands-on da aula

Parte A - testes

1. Rodar `yarn test` e observar a saída.
2. Abrir `src/__tests__/components/buttons/button.test.tsx`.
3. Identificar `render`, `screen`, `expect`, `fireEvent`.
4. Escolher um exercício:
 - (opção 1) duplicar a ideia de um `it` existente, mudando o texto e a asserção;
 - (opção 2) escrever um `it` novo que verifica `disabled` em um `Button` com a prop `disabled`.

Parte B - projeto

1. Revisar o fluxo da tela de Usuários (carregamento, tabela, botão).
2. Confirmar no navegador: menu `Administração > Usuários` e rota coerente.
3. Registrar em uma lista curta: **3 trechos** da tela (estado, efeito, render) e **1** trecho do service chamado.

Comandos de fechamento:

```
yarn test
yarn lint
yarn dev
```

9) Práticas integradas no projeto

As práticas abaixo reforçam os conteúdos estudados nas aulas anteriores, com foco em código de tela, navegação, estado e requests.

1. **Fluxo completo de listagem**
 - Abrir `src/screens/Administracao/Usuarios/index.tsx`;
 - identificar estado (`useState`), efeito (`useEffect`) e renderização da tabela;
 - explicar o caminho de dados: tela -> endpoint -> estado -> tabela.
2. **Navegação entre listagem e cadastro**
 - Confirmar a rota `/administracao/usuarios`;
 - validar o botão `Adicionar` com `useNavigate`;
 - percorrer no menu: `Administração > Usuários`.
3. **Leitura e ajuste de endpoint**
 - Abrir `src/services/endpoints/usuarios.js`;
 - mapear parâmetros usados em `pesquisarUsuarios` (`pagina`, `tamanho`, `ordemParaOrdenar`);
 - alterar apenas um parâmetro de consulta e validar o comportamento da tela.
4. **Ajuste de tabela com segurança**
 - alterar nome de uma coluna em `Usuarios`;
 - ajustar largura de coluna (`size`) para melhorar leitura;
 - validar se continua renderizando corretamente com `loading`.
5. **Tratamento de estado de erro e carregamento**
 - localizar `setCarregando(true/false)` e bloco `catch`;
 - confirmar mensagem de erro exibida via `toast`;
 - revisar se a tela volta para estado consistente quando a request falha.
6. **Prática de organização de código**
 - separar mentalmente o arquivo em blocos: `imports`, estado, efeito, tabela, `JSX`;
 - renomear variáveis locais que estejam pouco claras (sem alterar contratos);
 - validar com `yarn lint`.

Checklist de fechamento das práticas:

```
yarn lint
yarn dev
```

10) Erros comuns desta aula

1. **Teste quebrando por classe CSS** — o layout muda; prefira contrato de comportamento (texto, `disabled`, rótulo).
2. **Não isolar o que dá** — chamar API real no teste de componente; para telas com `fetch`, o próximo passo é `mock` explícito.
3. **Múltiplas asserções no mesmo it sem critério** — ok para fluxo pequeno; se o teste ficar opaco, dividir em mais de um `it`.
4. **Não rodar o teste após a alteração** — sempre `yarn test` no arquivo tocado.

11) Exercícios de fixação

1. Qual comando roda a suíte de testes no `frontend` ?
☐ `yarn build`
☒ `yarn test`
☐ `yarn preview`
☐ `node jest`
2. O que o `render` do React Testing Library faz?
☐ chama a API de produção
☒ monta o componente em um DOM de teste
☐ inicia o Vite
☐ compila TypeScript
3. Em um teste de clique, o que `jest.fn()` costuma representar?
☐ rota do React Router
☒ função colaboradora para verificar se foi chamada
☐ estado global
☐ `useEffect`
4. Onde costumam ficar arquivos `*.test.tsx` no projeto?
☐ `public/`
☒ `src/__tests__/...` (ou padrão equivalente do Jest)
☐ `node_modules/`
☐ raiz fora de `src`
5. O principal objetivo de testar um botão de salvar, nesta aula, é:
☐ medir desempenho de rede
☒ garantir que o clique dispara a ação esperada (ou o estado esperado)
☐ validar banco de dados
☐ substituir o `eslint`

12) Desafio para casa

1. Abrir de novo `src/__tests__/components/buttons/button.test.tsx`.
2. Escrever **um** novo caso de teste (novo `it`) que ainda não exista no arquivo.
3. Revisar `src/screens/Administracao/Usuarios/index.tsx` e anotar **um** `data-testid` visível e o que ele identifica na interface.
4. Rodar:

```
yarn test
yarn lint
```

Regras

- manter o escopo em um componente de teste só;
- não alterar o contrato público de serviços;
- se o teste novo falhar, ajustar o teste ou o componente, mas registrar o porquê.

13) Resumo final

Ao final desta aula, o aluno deve conseguir:

- explicar o papel de Jest e React Testing Library no `frontend` ;
- localizar testes em `src/__tests__` e executá-los com `yarn test` ;
- ler a estrutura de um `describe / it` e os principais `expect` ;
- relacionar a prática de teste com a leitura de tela e serviço da aula anterior;
- aplicar a metodologia da aula (arranjo, ação, asserção) em **um** caso novo.